

AMATH 582: Home Work 2

Mengze Yu, Email address: mengzy3@uw.edu

This report focuses on analyzing the kinematic characteristics of a novel robotic system. The robot exhibits three distinct modes of locomotion: walking, running, and jumping. Principal Component Analysis (PCA) will be employed to reduce the dimensionality of the collected motion data while preserving essential information, thereby facilitating subsequent analysis. Within this PCA-transformed space, a classification method will be developed through training on existing data to accurately determine the robot's motion state at any given time.

Introduction and Overview

Humanoid robot is a popular topic and worth observation and analysis. Therefore, we got some data from a robot which has a total of 38 joint nodes, and the position of these 38 nodes in space can be recorded at an interval of 1.4 seconds (100 timesteps) to obtain a set of data about the robot's motion. In this report, six sets of walking, running and jumping data were recorded, five of which were training sets and one was test set. An example image can be demonstrated:

For such a robot model, the ultimate goal of this paper is to design a clustering algorithm to ensure that the ongoing action of any given robot can be predicted by its current state, i.e. the coordinates of its 38 joints.

Theoretical Background

In this report, the Principal Component Analysis (PCA) method is mainly used for dimensionality reduction analysis and clustering. To construct a PCA basis, Singular Value Decomposition (SVD) is also strongly required.

To reduce the dimension of the centered data set X_c , a matrix A can be constructed and it can be expressed by the SVD as: $A = U\Sigma V^T$, where U is the Left Singular Vector, Σ is Singular Values and V^T is Right Singular Vector. What's more, it can be truncated to produce a basis for PCA space. To check how well the data set is approximated by the truncated A , the Frobenius norm:

$$\|A\|_F = \left(\sum_{i=1}^n \sum_{j=1}^m |a_{ij}|^2 \right)^{1/2}$$

can be used. It can be also equivalent to calculate the cumulative energy by $E_k = \frac{\sum_{i=0}^k \sigma_i}{\sum_{i=0}^r \sigma_i}$, where k is the dimension which PCA truncates, σ_i s are the diagonal elements of Σ and r is the rank of Σ .

In the generated PCA space, the information of the original data set will be provided by the projection matrix: $\tilde{X} = U^T X_{centered}$, and a centroid of each kind of motion can be easily found from the projection matrix and all the data can be classified by looking for the nearest centroid to it by Euclidean distance. Thus, for any given test data, its type can be predicted.

Further more, the the K-Nearest Neighbors (KNN) method was also used. Its goal is to find the nearest K points for the point we want to predict and judge its label by the label appears most frequently in its neighbor.^[1]

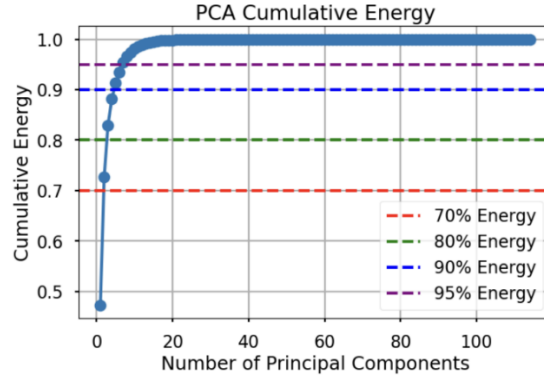
Algorithm Implementation and Development

In this report, numpy package has been used to process data, including taking the mean value and Data slicing. More than that, it can also compute the singular value matrix and norms with the help of linalg package. This provides the preparation for generating PCA spaces. After that, the sklearn packages are used to look for the centroids, train the model through PCA method and check the accuracy of the prediction. Finally, for the visualization, the matplotlib package is also included.

Computational Results

Since, the train data we have are 15 files of the record of the robot's single motions in during a fixed period of time, which is a data set shaped as 114*100. Therefore, we need to compile these data together and stack them in the time dimension, which leads to a 114*1500 matrix X_{train} . Such matrix is suitable for implementing the PCA method and it can follow that PCA modes are spatial modes and coefficients are time-dependent coefficients. However, since the SVD is required, a further centering is also needed, which means that we should take the mean in the time dimension and subtract the mean from the train matrix.

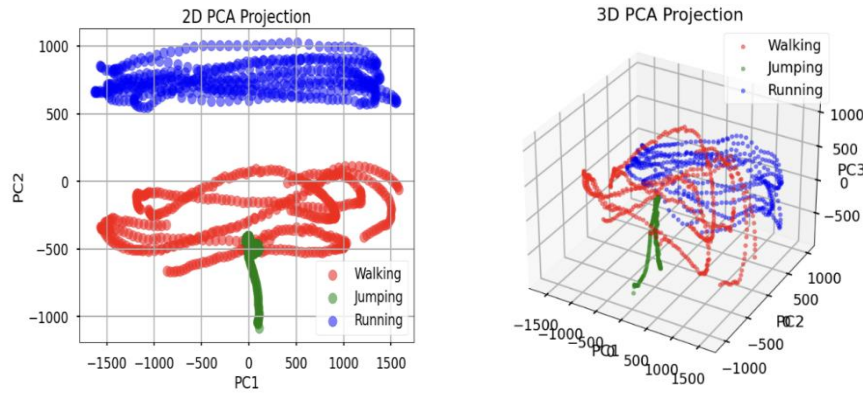
After processing the data, the SVD can be used on the centered train matrix. Then, the Left Singular Vector U , Singular Values Σ and Right Singular Vector V^T can be obtained. Since Σ is already known, the cumulative energy can be calculated by $E_k = \frac{\sum_{i=0}^k \sigma_i}{\sum_{i=0}^r \sigma_i}$. Through the value of the cumulative energy, its relationship with the dimension which PCA truncates k can be shown in the following figure:



The figure of cumulative energy with respect to k

Through the figure we can know that if we want to keep to approximate X_{train} up to 70% (80% , 90% , 95%) in Frobenius norm, 2 (3, 5, 7) PCA spatial modes are needed, which indicates that if we want the result of the prediction is reliable, at least 7 PCA spatial modes are needed.

The next step is to reduce the dimension by truncating the PCA modes and generate a new basis. In the new basis, the information of the original data set , as has been mentioned, can be provided by the projection matrix: $\tilde{X} = U^T X_{centered}$, its size is $k(\text{the truncated dimension}) \times 1500$. To visualize the generated basis we can plot the projection matrix in 2D and 3D coordinates and the axis for the plots are the first 2 and 3 trajectories:



The figure of 2D and 3D projection matrixes

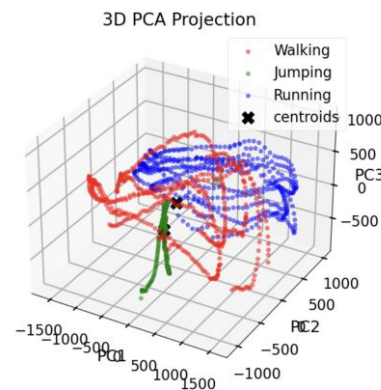
Through the figure, it's also obvious that the points for different motions will gather together in groups of their own and easy to distinguish, except for a few points of the jumping and walking. Such result also gives us some inspiration to design the classifier. By the way, the plots may have some few differences if we directly use the PCA function in Python, since the direction of the coordinates might be opposite, but the main result will remain the same.

After generating the new basis, we can design a classifier on the PCA space. The first thing we need to do is to create a vector of ground truth labels with an integer per motion, for instance, if the point comes from the walking motion, we

will label it as [0], and [1] for the jumping motion, [2] for the running motion in like manner. Since we loaded the motion in the order of walking, jumping and running, we can label the first 500 points as [0], next 500 points as [1], the final 500 points as [2].

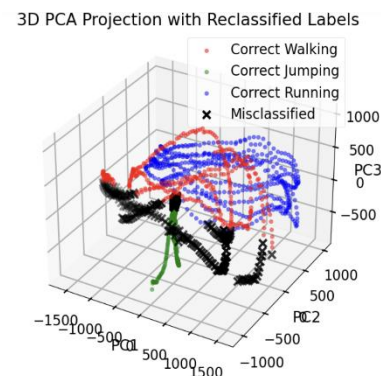
After that, the centroids can be found by calculating the mean of the points of different motions in each k-modes PCA space. For instance, the centroids in 3-modes PCA space is:

Centroids for each motion in 3-PCA space:
 Walking Centroid: [36.88211143 -253.35282541 175.91202104]
 Jumping Centroid: [23.88986635 -499.36826149 -72.5000755]
 Running Centroid: [-60.77197779 752.7210869 -103.41194553]



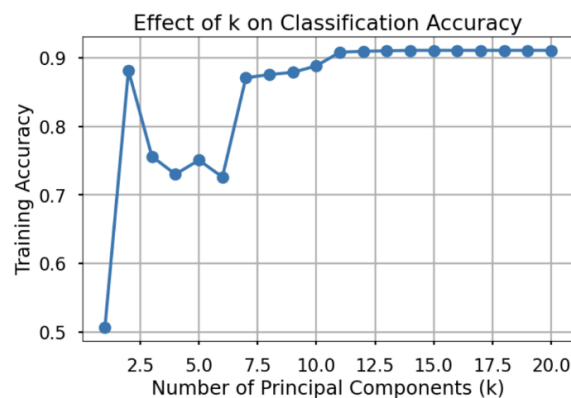
The figure of 3D projection matrixes and its centroids

Through the centroids, we can define the classifier: we can judge the label of a certain point by calculating the Euclidean distance of the point and each centroid and looking for the minimum one. Then the nearest centroid will show which motion the point belongs to and the conclusion will be recorded as a new label for the point. As an example, here's the figure of classified labels in 3-PCA space:



The figure of classified labels in 3-PCA space

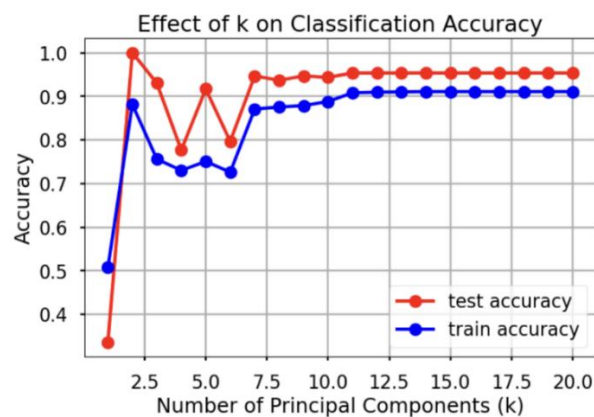
To test the effect of the classifier, we can try it in different truncated PCA space and plot the accuracy score in sklearn package:



The figure of training accuracies with respect to k

Through analyzing the figure, we can know that the training accuracy becomes really reliable when the number of the principal components are larger than 11, and can reach more than 90 percents. What's more, we can also find the highest accuracy appears at $k=14$, which indicates that 14 is the optimal number of principal components.

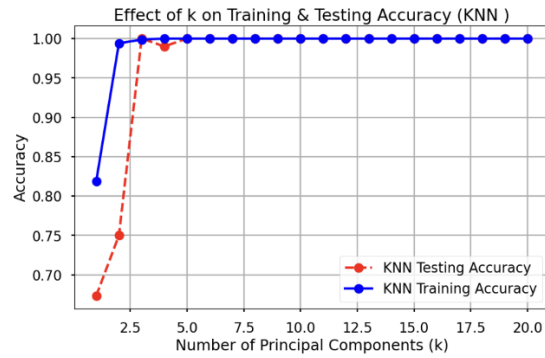
To further test whether this classification method is valid, we further perform the same analysis on the test set. Firstly, we need to reduce the dimension of the test set to k -PCA coordinates as well, which means that we centralized it with its mean and multiply it with the U we've already got. Let's call the processed matrix $\tilde{X}_{train}$ and its shape is $k(\text{the truncated dimension}) \times 300$. Also, we create a new label for the 300 points in $\tilde{X}_{train}$ and define the first 100 points as [0](walking), next 500 points as [1](jumping), the final 500 points as [2](running). After that, by calculating the Euclidean distance of each point and each centroid we can look for the nearest centroid for each point in the test set and gives a new label for it by the centroid. Finally, by contrasting the new label and the original label, we can got the accuracies of the method for a new sample. Here are some tries and results:



The figure of training accuracies and test accuracies with respect to k

Generally speaking, the accuracies tendency of the test accuracies is similar to the training accuracies. The method seems to be very reliable when the principal components are larger than 11, and the test accuracy can be even higher compared to the result of training set. Maybe that's because the test set is smaller and is more consistent with the centroids.

For a further discussion, we also implemented an alternative classifier: the K-Nearest Neighbors (KNN) method, that is to find the K (different from the k in k -PCA) nearest points for each point in the dataset and judge the label of each point by the most frequent label in its neighbour. We let $K=3$, and here's some results:



The figure of training accuracies and test accuracies with respect to k (KNN)

Through the figure we can find that the KNN method can ensure a higher accuracy and requires even less numbers of k -spatial nodes. Since, it's obvious that the accuracy started to reach near 100% when $k=3$.

Summary and Conclusions

To draw a conclusion, the PCA method is an effective method to reduce the dimension. In the report, we successfully used the PCA method by centralizing the training set of the motion of the robot and used the SVD method to construct projected matrixes. Also, we've shown how the number of PCA spatial modes affect the result of the to approximation. After that, we created a truth label for the training set and test set. To do some prediction, we found the centroids for each label through the projected matrix of the training set in k -PCA space. Then, we looked for the nearest centroids for each point in the training set and the test set and created a new label for them. By analyzing the result, we've found that the accuracy is high when the number of PCA spatial modes k is larger than 11, which means that the method we used for prediction is a reliable method. Also, we've tested the effect of an alternative classifier KNN, and found that it has an even stronger effect for prediction.

Acknowledgements

The author is thankful to Prof. Eli Shlizerman for the examples in class, which helped to design the SVD and PCA program. What's more, thanks should also be given to the Github Platform, which offered some good program examples.

References

- [1] Steinbach M, Tan P N. kNN: k-nearest neighbors[M]//The top ten algorithms in data mining. Chapman and Hall/CRC, 2009: 165-176.